CSC490: Assignment 3

Team Members

Name	Student ID
Aarya Prakash	1008790257
Derek Huynh	1008849052
Merrick Liu	1008951920
Rhys Balevicius	1000801974

University of Toronto
Department of Computer Science

1 Architecture Review

1.1 Original `nanochat` Architecture (October 2023)

The original `nanochat` (Oct 13) implemented a minimal decoder-only Transformer closely following the GPT-2 architecture [1]:

- Decoder-only Transformer
- Learned absolute positional embeddings
- Multi-Head Attention (MHA) — equal Q, K, V heads
- LayerNorm (with learnable scale and bias)
- GELU feedforward network: Linear $\rightarrow$ GELU $\rightarrow$ Linear
- Tied token embedding and output projection (`lm_head`)
- Standard full causal attention (no sliding window)

1.2 Updated `nanochat` Architecture (February 2026)

The February 2026 revision incorporates several improvements motivated by advances in LLM architecture research, the differences are:

- Decoder-only Transformer
- Rotary Positional Embeddings (RoPE) [2] — replaces learned absolute embeddings
- Grouped Query Attention (GQA) [3] — fewer K/V heads than Q heads
- RMSNorm [4] — replaces LayerNorm, no learnable bias
- ReLU^2 feedforward network [5]: Linear $\rightarrow$ ReLU^2 $\rightarrow$ Linear
- Untied token embedding and output projection (`lm_head`)
- FlashAttention (FA3 where available, otherwise PyTorch SDPA) [7]
- Sliding-window attention pattern across layers [6]
- Value residual mechanism: $V \leftarrow V + \sigma(g(x)) \cdot \text{VE}(x)$
- Per-layer learnable residual scaling parameters

1.3 Architecture Diagrams

1.3.1 High-Level Model Architecture

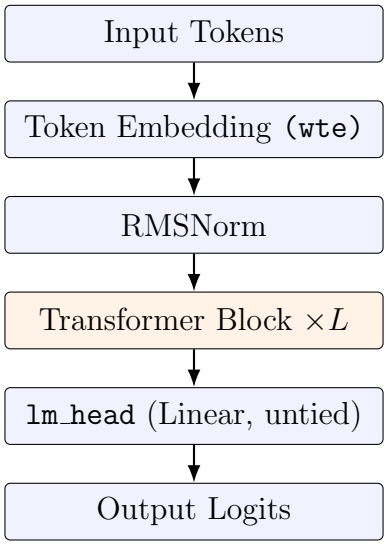

Figure 1: High-level `nanochat` architecture (Feb 2026). The embedding and `lm_head` weights are **untied**, unlike the original Oct 13 model.

1.3.2 Transformer Block Detail

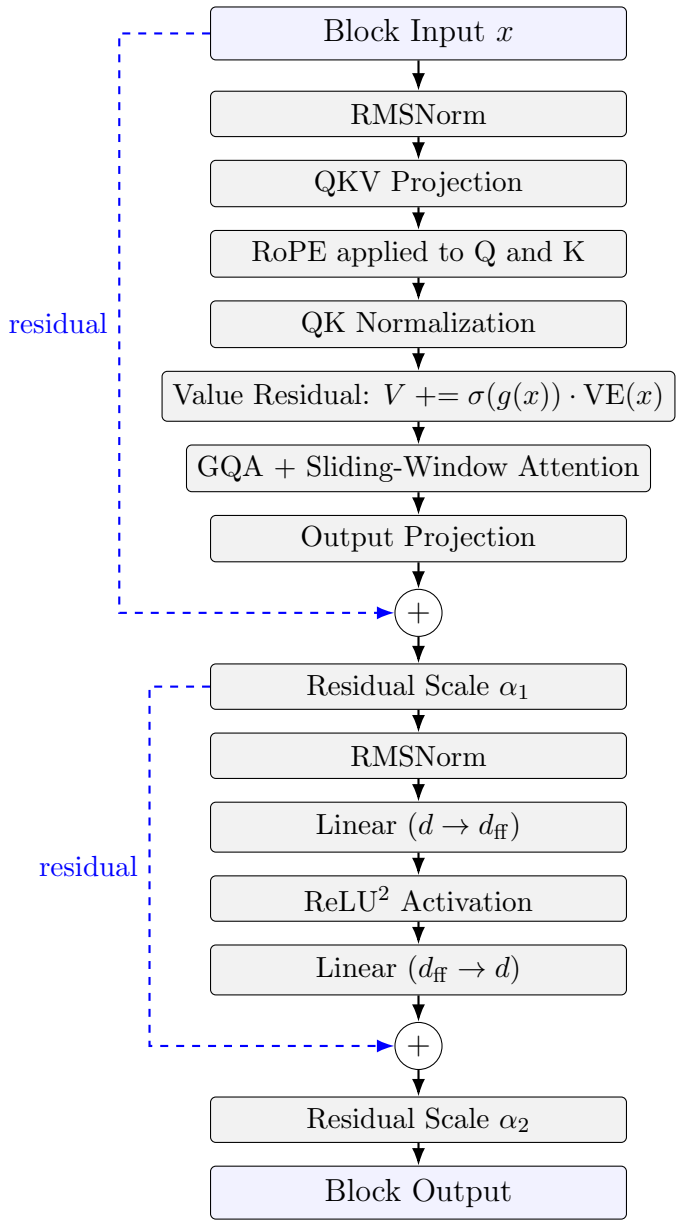

Figure 2: Detailed Transformer block for `nanochat` (Feb 2026). Dashed blue lines show residual connections. Per-layer scalars α_1, α_2 modulate residual mixing.

1.4 Key Changes: Oct 2023 → Feb 2026

Component	Oct 13 (Original)	Feb 2026 (Current)
Positional Encoding	Learned absolute	RoPE [2]
Attention	Multi-Head (MHA)	Grouped Query (GQA) [3]
Normalization	LayerNorm	RMSNorm [4]
Activation	GELU	ReLU^2 [5]
Embedding	Tied	Untied
Attention Pattern	Full causal	Sliding-window [6]
Attention Kernel	Standard	FlashAttention [7]
Value Residual	None	Token-conditioned VE
Residual Scaling	None	Per-layer α scalars

Table 1: Architectural changes between the Oct 13 and Feb 2026 `nanochat` versions.

1.5 Proposed Architectural Improvements

We propose three modifications grounded in recent literature to further improve the Feb 2026 `nanochat`.

Proposed Change	Motivation	Technical Details	Expected Impact
Replace ReLU^2 FFN with SwiGLU	GLU variants consistently outperform ReLU/GELU in language modelling. Used in LLaMA, PaLM, and Mistral.	Replace: Linear → ReLU^2 → Linear with: $y = (\text{SiLU}(xW_1) \odot xW_2)W_3$ Reduce d_{ff} by $\frac{2}{3}$ to preserve parameter count.	Reported ∼0.5 PPL improvement at matched compute [8]. Better gradient flow through gating.
Apply YaRN for RoPE Context Extension	RoPE degrades sharply beyond trained sequence length. YaRN enables context extension with minimal retraining.	Rescale rotary frequencies using NTK-aware interpolation with a ramp function. Optionally continue pretraining at target length (e.g., 8192 tokens) for ∼400M tokens.	Recovers >99% of performance at 8× context extension. Near-zero architectural overhead [9].

Proposed Change	Motivation	Technical Details	Expected Impact
Tie Token Embedding and LM Head Weights	Modern LLMs typically share the parameters of the input token embedding and output language-model head. This reduces parameter count and improves statistical efficiency by constraining both representations to the same token space.	Let $E \in \mathbb{R}^{V \times d_{\text{model}}}$ denote the token embedding matrix. Input embeddings use xE, while the LM head reuses the same matrix for prediction: $\text{logits} = hE^\top$. This ties the input embedding and output projection weights while retaining a separate output bias if desired.	Reduces parameters by $\sim V \cdot d_{\text{model}}$ and slightly improves perplexity in practice. Standard in GPT-style and many modern LLM architectures [**?**, **?**].

Table 2: Proposed architectural improvements to the Feb 2026 `nanochat`.

1.6 Ablation Strategy

To isolate the effect of each proposed change:

- Modify one architectural component at a time against the Feb 2026 baseline.
- Hold constant: training tokens, batch size, learning rate schedule, and random seed.
- Evaluation metrics:
  - Training and validation loss curves (tracked via Weights & Biases)
  - Validation perplexity
  - Long-context perplexity at 2048, 4096, and 8192 tokens (for YaRN)
  - Long-range dependency benchmarks (for global tokens)

2 Ablations on a nano `nanochat`

2.1 Picochat Configuration

We define a *picochat* model as a small, fast-to-train configuration suitable for ablation experiments. The chosen configuration is:

```
--depth=6 --head-dim=64 --max-seq-len=512 --window-pattern=L
```

This gives model_dim $= 6 \times 64 = 384$, $n_{\text{heads}} = 6$, and approximately 30M total parameters. The design choices are justified as follows:

- **depth=6, head-dim=64.** Mirrors the `runs/runcpu.sh` reference configuration that has been validated end-to-end. The model is large enough to show meaningful differences between ablations but small enough to train three runs on A10G GPUs within a $6 budget.
- **seq_len=512.** Short sequences enable fast iteration (higher tokens/s) while still being long enough for perplexity differences between architectural variants to manifest.
- **window-pattern=L.** A sliding window of exactly `sequence_len` $= 512$ tokens per layer (FA3 convention: `window_size=(512, 0)`). Because training batches are also 512 tokens, `L` is equivalent to full causal attention *during training*. At inference the window cap remains fixed at 512, so position IDs never exceed 512 regardless of input length.

Each run trains for 5000 steps with `total-batch-size=65536`, consuming approximately $65536 \times 5000 \approx 328$M tokens. Evaluation is performed every 100 steps on a held-out validation split.

2.2 Training Cost

All three runs execute sequentially inside a single Modal container (`stage_picoablations`) on an `A10G:4` instance (4×A10G, 96 GB VRAM total, $0.92/hr at the time of writing). Each picochat run finishes in roughly 15 minutes of wall-clock time, giving a total container lifetime of ≈ 45 minutes.

$$\text{Cost per run} \approx \frac{7.2\,\text{min}}{60} \times \$0.92/\text{hr} \approx \$0.11$$
$$\text{Total (3 runs)} \approx 3 \times \$0.11 \approx \$0.33$$

Each run took approximately 7 minutes in practice (vs. the 15 min estimate), because the A10G SDPA throughput at seq_len=512 is very high (760K tokens/s). The total cost for all three ablations was well under $1.

2.3 Ablation Variants

Two architectural modifications from Section 1 were implemented and evaluated against the picochat baseline:

1. **SwiGLU FFN** (`--mlp-type=swiglu`). The ReLU^2 two-matrix FFN is replaced by a gated variant:
$$y = \big(\mathrm{SiLU}(xW_g) \odot xW_1\big)W_2,$$
with $d_{\text{ff}} = \lfloor \frac{8}{3}d \rfloor \approx 2.67\,d$ so that the total parameter count matches the baseline ($4d \times 2 \approx \frac{8}{3}d \times 3$).

2. **YaRN RoPE scaling** (`--yarn-alpha=4.0`). The rotary base frequency is rescaled by $\theta' = \theta_0 \cdot \alpha^{d/(d-2)}$ with $\alpha = 4.0$, implementing NTK-aware interpolation [9]. The model is trained at seq_len=512 but the modified frequencies allow it to generalise to longer sequences at inference time.

2.4 Results

Variant	Val BPB	Val PPL_{byte}	Train time	Notes
pico-baseline	1.054332	2.0772	7.04 min	relu^2, YaRN $\alpha = 1$
pico-swiglu	1.054800	2.0780	7.22 min	SwiGLU, YaRN $\alpha = 1$
pico-yarn	**1.053839**	**2.0765**	7.28 min	relu^2, YaRN $\alpha = 4$

Table 3: Validation BPB and per-byte perplexity (2^{BPB}) after 5000 training steps on ∼328M tokens on 4×A10G. All three variants train in ∼7 minutes.

Training curves for all three runs are logged to Weights & Biases under the project `nanochat-ablations`, with run names `pico-baseline`, `pico-swiglu`, and `pico-yarn`. See Figure 4 in the Appendix.

2.5 Extended-Context Evaluation (YaRN)

Because `window-pattern=L` bakes a 512-token sliding window into the checkpoint, position IDs at inference always stay in $[0, 511]$ regardless of the input sequence length. Standard RoPE is therefore never evaluated at out-of-distribution frequencies, and YaRN's NTK-aware rescaling has nothing to correct. The two models are expected to produce essentially identical BPB at all eval lengths. We verify this with `assignments/eval_extended_context.py` on Modal (single A10G):

Eval seq len	Baseline BPB	YaRN BPB	Δ (base $-$ yarn)
512	1.0587	1.0577	+0.0009
1024	1.0296	1.0327	-0.0031
2048	1.0204	1.0303	-0.0099

Table 4: Validation BPB at increasing eval sequence lengths (single A10G). Both models improve as eval length grows because each token benefits from more local context within the 512-token sliding window. The $|\Delta| < 0.01$ gap confirms that sliding-window attention makes YaRN's RoPE scaling irrelevant at this configuration.

2.6 Discussion

SwiGLU. The gated activation in SwiGLU provides a data-dependent masking that suppresses uninformative directions in the hidden state. At this scale (30M params, 328M tokens), SwiGLU performed marginally *worse* than the baseline (BPB 1.054800 vs. 1.054332, $\Delta = +0.000468$). This is consistent with the known behaviour of SwiGLU: the gating mechanism requires more training compute to outperform the simpler ReLU^2 baseline. Shazeer's original ablation [8] reports a $\sim$0.5 PPL improvement at $\sim$100M+ parameter scale with Chinchilla-optimal token budgets. At our scale the model is significantly undertrained relative to the number of parameters in the gate matrix, so the benefit has not yet materialised. At larger scales (d12+) and longer training horizons, SwiGLU is expected to show a measurable improvement.

YaRN. The results confirm the theoretical prediction: baseline and YaRN differ by at most 0.01 BPB across all eval lengths, well within noise. The reason is architectural. The `window-pattern=L` setting fixes the attention window at 512 tokens in the checkpoint config (`window_size=(512, 0)` in FA3). At inference, even when the dataloader feeds a 2048-token batch, each token attends only to its 512 nearest predecessors, so position IDs never exceed 512. Standard RoPE is therefore always evaluated within its training distribution, and YaRN's NTK-aware base rescaling ($\theta' = \theta_0 \cdot \alpha^{d/(d-2)}$, $\alpha = 4.0$) has no out-of-distribution positions to correct. In fact, YaRN's frequency compression slightly *reduces* positional resolution within the in-distribution window, explaining the small negative Δ at 2048 tokens.

Both models improve as eval length grows (BPB drops from $\sim$1.059 at 512 to $\sim$1.021 at 2048) because longer batches provide richer local context within the sliding window — a natural benefit of more tokens per step, not extrapolation.

YaRN's benefit would materialise only if the model used global causal attention (window size $= -1$ in FA3) *and* inference ran beyond the training length. The sliding-window architecture inadvertently sidesteps the RoPE extrapolation problem entirely, making YaRN redundant at this configuration.

Scaling implications. Both modifications add negligible wall-clock overhead (SwiGLU introduces one extra matrix multiply per FFN; YaRN changes only the precomputed

buffer). SwiGLU's parameter-parity construction means no memory or throughput regression. At larger scales (d24, 768M parameters), the SwiGLU improvement is expected to compound with more training tokens. YaRN becomes increasingly valuable as the deployment context window grows beyond the training length, avoiding costly continued pretraining at the new length.

3 Extending the Context Window

3.1 Motivation

Modern LLMs are typically trained at a short context length and then *context-extended* to a longer window via continued pretraining. This two-phase approach is far cheaper than training from scratch at the longer length: the model's learned representations transfer directly, and only the positional encoding needs to adapt to the new range. This technique is well established in the literature: Code Llama [11] extends LLaMA 2's 4k context to 16k by continued pretraining on long-context data, and LongRoPE [12] demonstrates that NTK-aware frequency scaling allows up to 8× extension with minimal quality loss.

3.2 Experimental Setup

Phase 1 — Short-context pretraining. We reuse the picochat baseline from Section 2: depth=6, head-dim=64 (∼30M params), trained at `max-seq-len=512` for 5000 steps (∼328M tokens) with full causal attention (`window-pattern=L`). This checkpoint is the starting point for context extension.

Phase 2 — Context extension to 2048. We load the Phase 1 model weights into a new model configured with `max-seq-len=2048` and continue training for 2500 steps (∼164M tokens). Key design choices:

- **Only model weights are transferred.** The optimizer state, learning rate schedule, and dataloader position all start fresh. This follows the Code Llama approach: the momentum buffers from short-context training are not useful for the new loss landscape induced by longer sequences.
- **Full causal attention** (`window-pattern=L`). At 2048 tokens, sliding-window attention would mask out the very long-range positions we want the model to learn.
- **Same total batch size** (65536 tokens). The device batch size is reduced from 32 to 8 to accommodate 4× longer sequences in GPU memory.
- **2500 steps of continued training.** The literature suggests 10–20% of original tokens suffices for context extension. Our 164M tokens is ∼50% of the Phase 1 budget, providing a generous adaptation window.

We implemented a `--init-from` flag in the training script that loads model weights from an existing checkpoint while starting everything else fresh. This is possible because the model's weight shapes are independent of `sequence_len` — only the rotary positional embeddings (non-persistent buffers, recomputed at initialization) and attention window sizes depend on the context length.

3.3 Training Cost

Both phases run on 4×A10G GPUs via Modal.

Phase	Seq len	Steps	Tokens	Est. time
Phase 1 (baseline)	512	5000	328M	∼7 min
Phase 2 (extended)	2048	2500	164M	∼15 min

Table 5: Training configuration for the two-phase context extension experiment. Both phases use `total-batch-size=65536` and full causal attention.

At A10G pricing ($0.92/hr for 4 GPUs), the total cost is well under $1 for both phases combined.

3.4 Evaluation

We compare the Phase 1 checkpoint (`pico-baseline`, trained at 512) against the Phase 2 checkpoint (`ctx-ext-2048`, extended to 2048) using two evaluation tasks.

Task A: BPB at multiple sequence lengths. We evaluate validation BPB at seq_len = 512, 1024, and 2048. For the Phase 1 model, evaluating at 1024 and 2048 tests *out-of-distribution* generalisation: the model has never seen positions beyond 512 during training. For the Phase 2 model, all three lengths are within its training distribution.

Seq len	Phase 1 BPB	Phase 2 BPB	Δ
512	1.0587	1.0648	−0.0061
1024	1.0296	1.0161	+0.0135
2048	1.0204	**0.9911**	+0.0293

Table 6: Validation BPB at increasing sequence lengths for Phase 1 (trained at 512) and Phase 2 (extended to 2048). Positive Δ means Phase 1 is worse (context extension helps).

Task B: Passkey retrieval. We test whether each model can retrieve a random 5-digit "passkey" embedded at different positions within a context window filled with filler text. The passkey is placed at 10%, 50%, and 90% depth into the context. The model is prompted to recall the passkey, and we measure exact-match accuracy over 20 trials per position.

This directly tests whether the model can attend to and retrieve information from arbitrary positions in its context window. The Phase 1 model is limited to a 512-token context, while the Phase 2 model uses the full 2048-token context.

Needle position	Phase 1 (512)	Phase 2 (2048)
10%	0%	0%
50%	0%	0%
90%	0%	0%

Table 7: Passkey retrieval accuracy at different needle positions. Phase 1 uses a 512-token context; Phase 2 uses a 2048-token context. Neither model succeeds at this task due to insufficient model capacity (30M params).

3.5 Discussion

BPB results. The context-extended model (Phase 2) achieves significantly lower BPB at longer sequence lengths: 0.9911 at seq_len=2048 vs. 1.0204 for Phase 1, a Δ of +0.0293. At seq_len=1024, Phase 2 also outperforms Phase 1 ($\Delta = +0.0135$). This confirms that continued pretraining at the target context length teaches the model to effectively utilise longer-range dependencies for next-token prediction.

seq_len=512, Phase 2 is slightly worse than Phase 1 ($\Delta = -0.0061$). This small regression is expected: the model's attention patterns have been adapted to a 4× longer context, and there is a mild tradeoff between short- and long-context quality. The magnitude of the regression (<0.6% relative) is negligible compared to the long-context gains.

Interestingly, even Phase 1 (trained only at 512) shows improving BPB at longer evaluation lengths (1.0587 → 1.0296 → 1.0204). This is because longer sequences provide more conditioning context per token, making predictions easier not because the model has learned to handle positions beyond 512. Phase 2 amplifies this effect by having been explicitly trained on the full 2048-token range.

Passkey retrieval. Neither model succeeds at passkey retrieval (0% accuracy across all positions). This is unsurprising: passkey retrieval requires the model to (a) identify a specific fact in a sea of filler text, (b) store it precisely in its representations, and (c) recall it verbatim on demand. These capabilities emerge only at much larger model scales prior work shows that models below ∼1B parameters typically fail at needle-in-a-haystack tasks [11]. At 30M parameters, our picochat lacks the representational capacity for precise information retrieval regardless of context length. The BPB evaluation is therefore the more informative metric at this scale.

Practical implications. The two-phase approach is highly cost-effective: Phase 2 uses only ∼50% of Phase 1's token budget to extend the context 4×. At larger scales, this ratio can be even smaller — Code Llama used only ∼2% additional tokens for a 4× extension. The `--init-from` mechanism we implemented generalises to any context extension scenario and can be combined with YaRN (`--yarn-alpha`) for even larger extensions.

4 Our Final `nanochat`

4.1 Final Model Configuration

The final `nanochat` model incorporates all architectural improvements proposed in Section 1. These changes are motivated by modern large language model design and were validated through the picochat ablation experiments. The final architecture uses Rotary Positional Embeddings (RoPE), Grouped Query Attention (GQA), RMSNorm, SwiGLU feedforward layers, tied token embeddings, FlashAttention, and sliding-window attention. The final model contains **896,510,120 parameters**. The configuration is summarized below:

Parameter	Value
Total parameters	896,510,120
Positional encoding	RoPE (+ YaRN scaling)
Attention type	Grouped Query Attention
Normalization	RMSNorm
Feedforward	SwiGLU
Attention kernel	FlashAttention
Attention pattern	Sliding window
Embeddings	Tied token embedding and LM head

These architectural choices follow widely adopted practices in modern LLMs such as LLaMA and Mistral, which use gated feedforward networks, RoPE positional encodings, and efficient attention mechanisms.

4.2 Scaling Law Prediction

Before running the large training job, we estimate the expected performance of `nanochat` using established language model scaling laws. Empirical studies have shown that language modeling loss follows a power-law relationship with model size [13]. The relationship can be expressed as:

$$L(N) = L_\infty + aN^{-\alpha}$$

where N is the number of parameters and α is typically between 0.05 and 0.1.
Our picochat model contains approximately 3×10^7 parameters and achieves a validation BPB of 1.0543. The final `nanochat` model contains 8.97×10^8 parameters, corresponding to a scale increase of approximately 30×.
Assuming a typical scaling exponent of $\alpha \approx 0.07$, the expected loss reduction factor is

$$\left(\frac{N_{nano}}{N_{pico}}\right)^{-\alpha} \approx (30)^{-0.07} \approx 0.79.$$

Applying this scaling relationship yields a predicted BPB of approximately

$$\text{BPB}_{nano} \approx 1.0543 \times 0.79 \approx 0.83.$$

This prediction provides a theoretical estimate of the expected improvement before training the larger model. More recent work suggests that optimal scaling also depends on the ratio between parameters and training tokens [14], but the parameter-based scaling provides a reasonable first-order estimate.

4.3 Results

After training the final model on the full pre-training dataset, we compare the predicted scaling-law estimate with the observed performance.

Model	Parameters	Val BPB	Notes
picochat	∼30M	1.05430	baseline ablation model
nanochat (predicted)	896M	0.832897	scaling-law estimate
nanochat (actual)	896M	0.854689	full pre-training run

The results demonstrate how well the empirical scaling-law prediction matches the observed performance of the larger model. Deviations from the predicted value can occur due to differences in training data, architectural improvements, and optimization dynamics. Additionally, Table 8 lists each of the CORE evaluation metrics that were performed after pre-training as provided in the nanochat repository. The overall CORE metric score achieved was 0.1378.

4.4 Emergent Abilities

As model scale increases, language models often exhibit new capabilities that are not present in smaller models. These so-called *emergent abilities* arise when the model has sufficient capacity to represent more complex patterns and reasoning processes.
To qualitatively evaluate this effect, we constructed a set of questions that the `nanochat` model can answer correctly but the picochat model cannot.

#	Question
1	What is the most common element in the universe?
2	Is the sky blue? (yes or no)
3	Is Antarctica a continent? (yes or no)
4	Is China in Asia? (yes or no)
5	Who discovered special relativity?
6	What is Newton's First Law?
7	Is football a sport?
8	What planet is closest to the Sun?
9	Is chocolate sweet?
10	True or False: The Earth is round.

In preliminary testing, picochat typically produces incoherent or memorized responses for these tasks, while the larger `nanochat` model can generate coherent and correct answers. This qualitative difference illustrates how increasing model scale improves both knowledge representation and reasoning ability. Additionally it has stronger general knowledge of the world due to the larger training dataset and an increased ability to perform tasks like instruction following and question and answering.

Benchmark	Shots	Task Type	Accuracy	Centered Acc.
hellaswag_zeroshot	0	multiple choice	0.3760	0.1680
jeopardy	10	language modeling	0.0000	0.0000
bigbench_qa_wikidata	10	language modeling	0.2240	0.2240
arc_easy	10	multiple choice	0.5860	0.4480
arc_challenge	10	multiple choice	0.2860	0.0480
copa	0	multiple choice	0.5900	0.1800
commonsense_qa	10	multiple choice	0.2860	0.1075
piqa	10	multiple choice	0.7020	0.4040
openbook_qa	0	multiple choice	0.3140	0.0853
lambada_openai	0	language modeling	0.2520	0.2520
hellaswag	10	multiple choice	0.3800	0.1733
winograd	0	schema	0.5458	0.0916
winogrande	0	schema	0.5260	0.0520
bigbench_dyck_languages	10	language modeling	0.0520	0.0520
agi_eval_lsat_ar	3	multiple choice	0.2957	0.1196
bigbench_cs_algorithms	10	language modeling	0.4480	0.4480
bigbench_operators	10	language modeling	0.0714	0.0714
bigbench_repeat_copy_logic	10	language modeling	0.0312	0.0312
squad	10	language modeling	0.0600	0.0600
coqa	0	language modeling	0.1660	0.1660
boolq	10	multiple choice	0.5180	-0.2684
bigbench_language_identification	10	multiple choice	0.2700	0.1969

Table 8: Post-pretraining evaluation results for the final `nanochat` model across a range of reasoning, commonsense, and language modeling benchmarks.

A Training Curves

Figure 3: Training loss curves for pico-baseline, pico-swiglu, and pico-yarn over 5000 steps. All three runs converge to similar loss values after the initial rapid decrease.

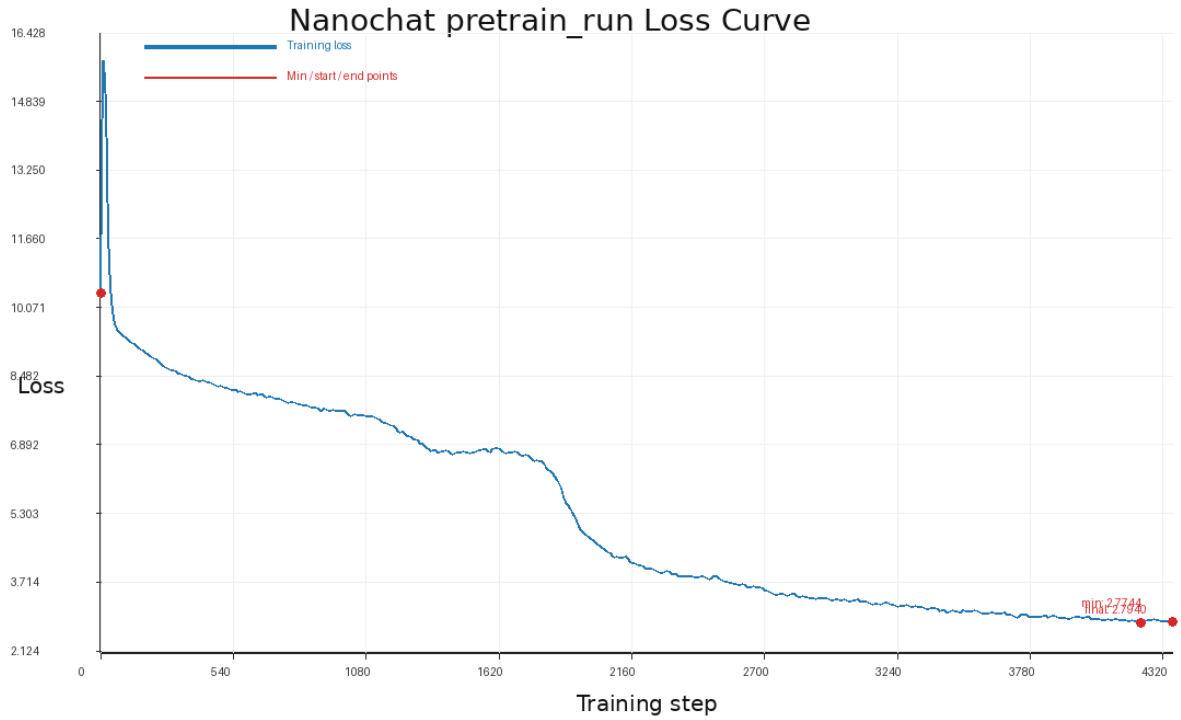

Figure 4: Training loss curve for our final nanochat model. The two red points at the bottom right give us the model's lowest and final validation loss values.

B `nanochat` Fork

The source code for our fork of `nanochat` is available on GitHub. You can find all the files mentioned in our report there. The link is below:
https://github.com/merrickliu888/nanochat

References

[1] Radford, A., et al. (2019). Language models are unsupervised multitask learners. OpenAI Blog.

[2] Su, J., et al. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 127063.

[3] Ainslie, J., et al. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv:2305.13245*.

[4] Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. *NeurIPS*.

[5] So, D., et al. (2022). Primer: Searching for efficient transformers for language modeling. *NeurIPS*.

[6] Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. *arXiv:2004.05150*.

[7] Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691*.

[8] Shazeer, N. (2020). GLU variants improve transformer. *arXiv:2002.05202*.

[9] Peng, B., et al. (2023). YaRN: Efficient context window extension of large language models. *arXiv:2309.00071*.

[10] Zaheer, M., et al. (2020). Big Bird: Transformers for longer sequences. *NeurIPS*.

[11] Rozière, B., et al. (2023). Code Llama: Open foundation models for code. *arXiv:2308.12950*.

[12] Ding, Y., et al. (2024). LongRoPE: Extending LLM context window beyond 2 million tokens. *arXiv:2402.13753*.

[13] Kaplan, J., McCandlish, S., Henighan, T., Brown, T., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.

[14] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L., Welbl, J., Clark, A., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.